Data Retrieval and Manipulation

PAGE NO.
DATE / /

- Aggregation / Aggregation:

Use of aggregate functions:

Count (column_name)

Max (Column_name)

Min (column_name)

SUM (column_name)

AVG (column_name)

Select with / Aggregate functions Syntax

```
SELECT count (column_name)
FROM   table_name
Where  conditions.
```

Count Aggregate Functions

1) Select count (emp_id) as Total_employee From employee;

output

Total_employee
24

Note: we can also use *, count (*) shows all result.

dnawadolie ghising

PAGE NO.....
DATE / /

2) select count (distinct dept-name) as Department from employee;

output

Department
10

Max Aggregate Functions

1) select max (salary) as highest-salary from employee;

output

highest-salary
100000.250

Min Aggregate Functions.

1) select Min (salary) as lowest-salary from employee;

output

100000.250

Why Are the Results the same?

→ The results are the same because there is no variation in the [salary] column. When every row in the column has the same value, both [Max] and [Min] will naturally return that value.

chawla dorje arising

PAGE NO.....
DATE / /

Sum Aggregates functions.

1) seleeet sum(salary) as total_expanse from employee;

	total_expense
▶	1000004.450

AVG Aggregate functions

1) seleet Avg(salary) as Avarage_salary from employee;

	Avereage_salary
▶	41666.8520

dhawa dorje ghising

PAGE NO.....
DATE / /

Group data with GROUP BY and Filtering groups using HAVING

Group by Syntax

Purpose of Group by

→ The Group By clause groups rows in your dataset based on the values in one or more columns. It's often used with aggregate functions (like SUM, COUNT, AVG, etc) to calculate summary statistics for each group.

Task: calculate Total Spending Per customer Per Market Date

without Group by,

```
Select market-date, customer-id, price
from vg-sales
order by market-date, customer-id;
```

Result: This query lists individual rows of purchases showing duplicates for customers who made multiples purchases.

Grouping Results by Market-date and customer-id

To summarize the total spending per customer on each market date, use Group by with the sum Aggregate function:

```
Select
    Market-date,
    customer-id,
    sum(price) as total-spent.
from sales.
```

dhanraj dorje ghising

PAGE NO.....
DATE / /

Group by market_date, customer_id
Order by market_date, customer_id

Explanation

1) `Group by market_date, customer_id`

Groups all rows with the same `market_date` and `customer_id`.

2) `sum(price)`

for each group, calculates the total amount spent (price).

3) `Order by market_date, customer_id`;

Sorts the output in chronological order of market dates and by customer IDs.

Alternative Using Distinct.

If you're only interested in unique combinations of `market_date` and `customer_id` without any aggregation.

```
Select Distinct market_date, customer_id
from veg_sales
order by market_date, customer_id;
```

drawa dorie ghising

PAGE NO.....
DATE / /

Displaying Group Summaries:

1. Count the no. of items Purchased (Per Customer, Per Market Date)

```
Select
    Market_date,
    customer_id,
    count(*) As items_purchased.
From  fruit_sales
Group by  market_date, customer_id.
Order by  market_date, customer_id.
```

Explanation

- Group data by `market_date` and `customer_id`
- `count(*)` counts the no. of rows (purchase made by the customer on that date).

2. Sum the Quantities Purchased (Per Customer, Per market Date).

```
Select
    market_date,
    customer_id,
    SUM(quantity) as total_quantity
From  Fruit_sales.
Group by  market_date, customer_id.
Order by  market_date, customer_id
```

Explanations

- SUM(quantity) adds up the quantities purchased for each group.

PAGE NO.....
DATE / /

3) Count the no. of distinct Product Purchased (Per Customer, Per Market Date).

```
Select
    market_date,
    customer_id,
    Count(Distinct product) As unique_product
From ~~veg_sales~~ veg_sales
Group by market_date, customer_id
Order by market_date, customer_id;
```

Explanations:

- count(Distinct product) counts the unique product purchased in each group.

4) Combine Multiple summaries in a Single Query.

```
Select
    market_date,
    customer_id,
    sum(quantity) as total_quantity,
    count(Distinct product) as unique_product
From fruit_sales
Group by market_date, customer_id
order by market_date, customer_id.
```

Explanations:

- Combines SUM(quantity) and count(Distinct) in one query.

PAGE NO.
DATE / /

#Calculate total Spending Par Customer on Each day.

buyer

customer_id	first_name	last_name
1	Alice	Brown
2	Bob	Smith
3	Charlie	Johnson

Foreign key

Foreign key

Supplier

vendor_id	vendor_name
1	Fresh Produce
2	Daily Delights
3	Baked Goods

Primary key

Kinamui Primary key

Primary key

purchase_id	customer_id	vendor_id	purchase_date	Quantity	Cost per unit
1	1	1	'2025-01-20'	5	2.50
2	1	2	'2025-01-20'	2	3.70
3	2	1	'2025-01-21'	4	2.50
4	3	3	'2025-01-21'	3	5.00
5	1	3	'2025-01-22'	6	4.00
6	2	2	'2025-01-22'	1	3.75

dhavaldoje dhising

PAGE NO.....
DATE / /

calculate Total Spending per Customer at Each Vendor

we calculate the total spending of each customer on a given day by multiplying [quantity] and [cost_per_unit] for each row, then summing the results per customer per day.

```
Select
    customer_id,
    purchase_date,
    SUM (quantity * cost_per_unit) AS total_spent
From kirana_mail
Group by customer_id, purchase_date
Order by customer_id, purchase_date;
```

output

Customer_id	purchase_date	total_spent
1	2025-01-20	20
1	2025-01-22	24
2	2025-01-21	10
2	2025-01-22	3.75
3	2025-01-21	15.00

dhawaoarjesingh

PAGE NO.....
DATE / /

Total Spending Per Customer at Each Vendor.

-) Here, we calculate how much each customer has spent at each vendor, regardless of the date.

```
Select
    customer_id,
    vendor_id,
    SUM (quantity * cust_per_unit) As total_spent
from kinamail
Group by customer_id, vendor_id
order by customer_id, vendor_id;
```

Output

customer_id	vendor_id	total_spent
1	1	12.50
1	2	7.50
1	3	24.00
2	1	10.00
2	2	3.75
3	3	15.00

dhawa dorje ghising

PAGE NO.....
DATE / /

Total spending by Each Customer.

-> This query sums up the total amount spent by each customer across all vendors and dates.

```
Select
    customer_id,
    SUM(quantity * cost_per_unit) As total_spent
from kinamuil
Group by customer_id;
```

output

customer_id	total_spent
1	44.00
2	13.75
3	15.00

dhawa dorje ghising

PAGE NO.....
DATE / /

Detailed Report : Customer and vendor Details.

```
Select
    b.first_name,
    b.last_name,
    s.vendor_name,
    sum (k.quantity * k.cost_per_unit) As total_spent
from buyer b
left join kingmail k
    on b.customer_id = k.customer_id
left join supplier s
    on k.vendor_id = s.vendor_id

Group by b.first_name, b.last_name, s.vendor_name
order by b.last_name, s.vendor_name
```

How, (select b.first_name, b.last_name), it is normal to fetch b.first_name, b.last_name from buyer b but s.vendor_name & sum (k.quantity * k.cost_per_unit) are from outer table means not from buyer table. how that possible?

-) The reasons it works is due to JOINs and the concept of a combined dataset that is created during the execution of the SQL query.

How JOINs Create a Combined Dataset.

When you write a query that includes **Joins**, the database combines the rows from the specified tables (in this case, buyer, kingmail and supplier) into a single, virtual dataset for the duration of the query.

PAGE NO.....
DATE / /

Here's what happens in this query:

1. by buyer b.

- The query starts with the buyer table (aliased as b) creating an initial result set of all the rows from this table.

2) left join kinamail k on b.customer_id = k.customer_id.

- This step combines the rows from the [buyer] table with the rows from the [kinamail] table, based on the condition [b.customer_id = k.customer_id]

3) left join supplier s on k.vendor_id = s.vendor_id.

→ After the join, the resulting dataset will include columns from both tables (buyers and kinamail)

→ Next, the query joins the [suppliers] table with the intermediate result set from the previous step, using the condition, [k.vendor_id = s.vendor_id].

→ The result now includes columns from all three tables. ([buyer] [kinamail] and [supplier]).

Why You can use Columns from Other tables in SELECT

Once the [Joins] have created this combined dataset, the query has access to all columns from all the joined tables. That's why the following is possible

- [b.first_name] and [b.last_name] : These comes from the [buyer] table

PAGE NO.....
DATE / /

- `S.vendor_name` : This comes from the `Supplier` table

- `SUM(k.quantity * k.cost_per_unit)` : This is a calculation based on columns from the `Kinamail` table

The database doesn't care whether a column originates from the original `From` table (`buyer` in this case) one of the joined tables (`Kinamail` or `Supplier`). As long as the column is available in the combined dataset, it can be used in the query.

Example walk-through.

buyer table:

customer_id	first_name	last_name
1	Alice	Brown
2	Bob	Smith

Kinamail table.

~~Supplier table~~

purchase_id	customer_id	vendor_id	quantity	cost_per_unit
101	1	201	5	2.50
102	1	202	2	3.75

Supplier table.

vendor_id	vendor_name
201	Fresh Produce
202	Dairy Delights

PAGE NO.....
DATE / /

When the query is executed:

1. The `From` starts with all rows from `buyer`

2. The first `Left Join` combines the ~~customer~~ `buyer` table with the `Kinamail` table

3. ↳ For Alice (customer_id = 1), two rows are added. (one for each purchase)

↳ For Bob (customer_id = 2), no rows are added because he has no purchases.

3) The second `Left Join` combines the result with the ~~vendors~~ `Supplier` table.

- Each purchase is matched to its corresponding supplier (vendor).

The combined dataset will look like this:

first_name	last_name	vendor_name	quantity	cost_per_unit
Alice	Brown	Fresh Produce	5	2.50
Alice	Brown	Dairy Delights	2	3.75

Now, the `Select` statement pulls data from this combined dataset.

↳ b. first_name and b. last_name : Alice Brown.

↳ s. vendor_name : Fresh Products or Dairy Delights

↳ SUM (i. quantity * i. cost_per_unit)
Total spending for each vendor.

chavadorje.github.io

PAGE NO.....
DATE / /

output

first_name	last_name	vendor_name	total_spent
Alice	Brown	Baked Goods	24.00
Alice	Brown	Dairy Delights	7.50
Alice	Brown	Fresh Produce	12.50
Charlie	Johnson	Baked Goods	15.00
Bob	Smith	Dairy Delights	3.75
Bob	Smith	Fresh Produce	20.00

dhawa dorje ghising

Filtering with Having

PAGE NO.....
DATE . / /

1) Purpose

↳ The `Having` clause is used to filter data after aggregation. It works with grouped data, applying conditions on the aggregated values (e.g. `sum`, `count`, `Avg`).

↳ It differ from `where`, which filter before aggregation on raw data rows.

2. Typical Use cases

↳ Use `HAVING` to restrict grouped results based on the value of aggregate functions (e.g. filtering groups with a total count or sum above a certain threshold).

sales_data

Region	sale_id	sale_date	sale_amount
North	1	2024-01-10	5000.00
South	2	2024-01-11	4000.00
East	3	2024-01-12	8000.00
North	4	2024-01-13	7000.00
west	5	2024-01-14	3000.00
South	6	2024-01-15	12000.00
North	7	2024-01-16	6000.00
East	8	2024-01-17	4000.00
West	9	2024-01-18	9000.00
varchar(50)	Int Primary key	Date	Decimal

dhawa dorje dhising

PAGE NO.....
DATE / /

Sql code

```
Select
    region,
    Count(sale_id) As total_sales,
    Sum(sale_amount) As total_revenue,
From sales_data
Group by region
Having total_revenue >= 15000
Order by total_revenue Desc;
```

Output

region	total_sales	total_revenue
North	3	18000.00
South	2	16000.00

dhawa dorje ghising